Building a Mini GPT from Scratch

Findings & Analysis

Character-level decoder-only Transformer · trained on Shakespeare (balcony scene) · PyTorch

1. Overview

This report accompanies the notebook **Mini_GPT_From_Scratch.ipynb**. A GPT-style, decoder-only Transformer was implemented entirely from scratch in PyTorch (only `nn.Linear`, `nn.LayerNorm`, `nn.GELU` and `nn.Embedding` plus standard tensor ops — no `nn.Transformer`, `nn.MultiheadAttention`, or HuggingFace models). It was trained at the character level to predict the next character, then used to generate text under three decoding strategies: greedy, temperature, and top-k.

2. Data Preparation

The corpus is the *Romeo and Juliet* balcony scene (1,738 characters). Characters were tokenized at the character level, giving a vocabulary of **49** unique symbols with `char_to_idx` / `idx_to_char` lookup tables. A sliding window of length **128** produces autoregressive (input, target) pairs where the target is the input shifted one character to the left, yielding **1,609** training examples.

Corpus size	1,738 characters
Vocabulary size	49 characters
Sequence length (context)	128
Training examples	1,609

3. Model Architecture

The model stacks token + learnable positional embeddings, two decoder blocks, a final LayerNorm, and a linear head over the vocabulary. Each decoder block uses pre-LayerNorm, residual connections, manual multi-head masked self-attention (4 heads, causal mask), and a `Linear` → `GELU` → `Linear` feed-forward network.

Total parameters	425,777
Hidden dimension (d_model)	128
Attention heads	4 (head dim = 32)
Decoder layers	2
Feed-forward width	512 (4 x d_model)
Positional encoding	Learnable embedding

4. Training

The model was trained with the **Adam** optimizer (learning rate **1.5×10^{-3}**), batch size **32**, and cross-entropy loss for **8 epochs**. The learning rate was tuned so the loss falls steadily to a moderate

value rather than collapsing the next-character distribution to a single spike — the residual uncertainty is what makes the temperature and top-k comparisons meaningful.

Epoch	1	2	3	4	5	6	7	8
Loss	2.594	2.148	1.991	1.739	1.408	1.097	0.833	0.615

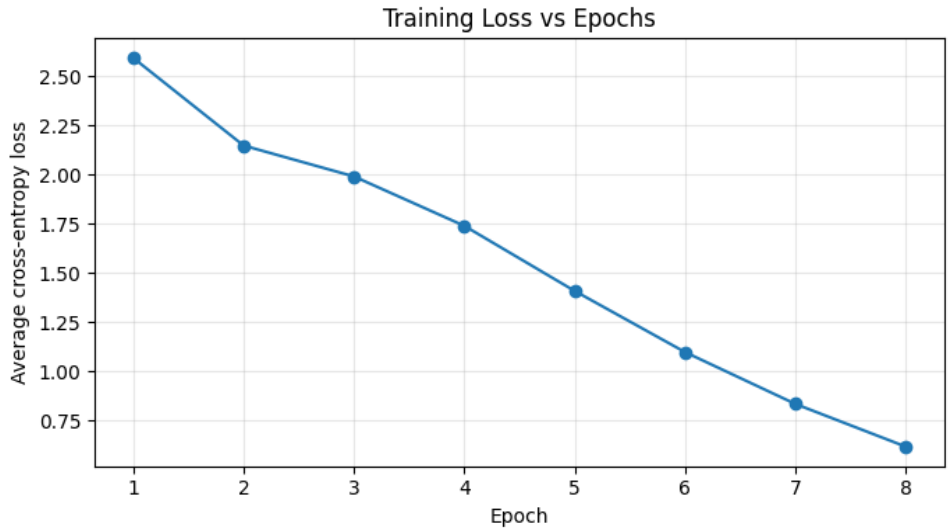

Figure 1. Training loss decreases smoothly and monotonically from 2.59 to 0.62 over 8 epochs.

5. Decoding Results (prompt: “ROMEO:”)

All samples below are the model's actual output for the prompt **“ROMEO:”**, taken directly from the executed notebook (excerpts).

Greedy decoding

```
ROMEO:
I know tame to to tell thee whee whee who I am:
My name, dear saint, is hateful to myself,
Because it is en is en the envis mousion,
Who mo sindo eat sindo alreareasco asen,
Twou it is ale in ale is ther sun.
Be sing an.
I is mo mall tong oo bone ot ot oteast.
What me of thy do nar that
Tam, ...
```

Temperature sampling

temperature = 0.5

```
ROMEO:
O:
By a ka kne
I know ame tot to to tell thee whee who I am:
My name, dear sar sainteful to myself,
Because it is en then emy the;
Had ent I I whill wer be wot.
Mon nore, tor bore, shat, nor shateateat,
Nor is nor sheaks is nor sall ...
```

temperature = 1.0

```
ROMEO:
O:
By a ka ow tow thell theca rose
By any other word woull sword sme
So Romeo would, wo werear Romeould,
Re heat whin t thow athitlt thee woug:
Wtat to OMy Mond, a all my nameatear is?
Bewor sit itthe wor editleas e sheey sphes
Twou ...
```

temperature = 1.5

```
ROMEO:
O:
Shall I ome all ad, be I’ll thy re re nor name, bapt sher r my shed;
He I hou t, wis tof it tot,
unoot t f Romeo at why wheldeaithe ender I wheHld no I I hee.
T:
My sal namell to is ttame.
:
’n to love she inger shao but bescoest ...
```

Top-k sampling

top-k = 5

```
ROMEO:
O:
By a ka kne
I know ame
I tort it meouhy wo, dof thor ris ar m, shaind sill the envious my ady,
The remou faid thesineesem, she is e envioresstall lives my busick en’d
gre but dole hest is do entus do hesco ent it it ent her es
To ...
```

top-k = 20

```
ROMEO:
O:
By a ka ow tow thell theca rose
By any other word woull sword sme
So Romeo would, wo werear Romeould,
Re heat whin t thow athitlt thee woug:
Wtat to OMy Mond, a all my nameatear is?
Bewor sit itthe wor editleas e sheey sphes
Twou ...
```

6. Analysis: Comparing the Decoding Strategies

Greedy decoding

Greedy decoding is **deterministic**: at every step it picks the single highest-probability character. Its output is the most faithful to the training text and reads as the most **coherent**, but because it can never deviate from the top choice it falls into short **repetition loops** (visible above as doubled fragments such as “to to tell thee” and “whee whee”). Its diversity is essentially **zero** — the same prompt always produces the same continuation.

Temperature sampling

Temperature T rescales the logits before softmax, acting as a smooth *coherence* ↔ *diversity* dial. At **T = 0.5** the distribution is sharpened, so output stays close to greedy: mostly coherent, low diversity. At **T = 1.0** the model samples from its true distribution, balancing structure and variety. At **T = 1.5** the distribution flattens, unlikely characters are chosen far more often, and the text becomes the **most diverse but least coherent**, drifting into malformed words.

Top-k sampling

Top-k keeps only the k most likely characters, then samples from that shortlist — retaining the variety of sampling while cutting off the unlikely tail that produces nonsense. A **small k (5)** is a tight, safer shortlist with moderate variety; a **larger k (20)** admits far more candidates and behaves close to plain T = 1.0 sampling, i.e. noticeably more diverse with more errors.

Conclusions

Most coherent	**Greedy decoding** (closely followed by low-temperature and small-k sampling) — it always commits to high-probability characters, at the cost of repetition and no variety.
Most diverse	**High-temperature sampling (T = 1.5)** — flattening the distribution maximises the chance of unusual characters, at the cost of coherence; large-k sampling is next.
Best balance	**Top-k with a modest k** (or temperature near 1.0) — it preserves variety while top-k's tail-cutting keeps the output from degenerating into noise.